

Mathematical modelling of personalised learning and learning trajectories

Introduction

It is impossible to imagine a modern education system without infrastructure, many components of which bear the prefix ‘electronic’ or ‘digital’ (electronic didactics, digital pedagogy, electronic lectures, etc.). Computer-assisted learning continues to evolve alongside the development of information technology. Support for constructivism and e-didactics, as well as the approach to didactic engineering, is outlined in [1, 2]. Rozitsa Doneva and her colleagues [3] compare traditional and e-learning in terms of the key elements of learning. The paper describes the advantages and disadvantages of several learning platforms (Moodle, LAMS, PeU) and presents the main functional capabilities of the BEST system, which incorporates all the positive aspects of the previous ones.

The modern approach to learning views learning as the construction of knowledge and incorporates the concept of formal knowledge representation, which has long existed as a major branch of artificial intelligence [4, 5]. One of the rapidly developing areas of knowledge representation (modelling) in computing is semantic networks [6, 7]. Like other information technologies, semantic networks are also applied in education. The work by Kristanson and his colleagues [8] outlines a solution to the problem of identifying knowledge gaps in textbooks, using the example of 10 linear algebra textbooks based on semantic networks.

Downs [9] describes learning objects with metadata to support reusability and accessibility for learning software agents. He also advocates an approach in which the creation of an e-course, much like software development, should be structured in accordance with:

- 1) the RAD concept, where a product is developed using reusable components. An online course is viewed as a system constructed from reusable learning objects;
- 2) the concept of object-oriented programming, where learning objects are created on the basis of a particular class within a class hierarchy and may be related (through association, aggregation, etc.) to other objects — such as test objects, course objects, and so on;
- 3) the concepts of open standards (LOM, SCORM), so that distributed learning environments and learning objects, developed by different authors in different languages and on different platforms and located on different computers, can work together [16].

The focus is primarily on course creation; semantic information (metadata) is used to integrate learning objects into an e-course and to describe its hierarchical structure. The formalism proposed for describing metadata — as in IMS and SCORM [13] — is XML, XSL and CSS.

The article by Arnold Kiva and his colleagues [14] discusses a semantic learning network, the algorithm for its construction and its visualisation. An algorithm is presented

for defining the logical sequence between the main concepts of a subject. In particular, semantic networks are examined for the subjects 'Economic Cybernetics', 'Algorithms and Programming' and 'Computational Mathematics'. The topological characteristics of the network are discussed on the basis of properties such as degree, centrality, clustering, etc.

The modern global web (Web3) incorporates a semantic component and is referred to as the Semantic Web [15]. Many educational systems have been developed based on this type of network and are referred to as the Semantic Learning Web. The main principles governing the operation of such systems are described in the works of Stutt, Motta and Devežik [10, 11].

Standards have been developed to integrate a learning course and a learning platform created by an educator and a programmer who are not familiar with one another, in order to ensure the interactivity of the course and obtain the necessary feedback. The SCORM standard describes the relevant mechanisms in detail [13]; these have been refined as technology has advanced [18].

This article represents an attempt at mathematical modelling of the process of personalised learning [19], which, in turn, is a constituent element of constructivism [20].

Based on the model described below, it is possible to specify a software system for machine-assisted intelligent analysis of the learning process (trajectory) — to identify rules (associations), similarities, differences and anomalies in the teaching methods used and in learners' learning trajectories [21, 22].

When didactically adapted learning material can be tailored to the learner's needs in order to improve the quality (bringing the learning outcome closer to the didactic objective) and efficiency of learning (in terms of time and other resources), this is referred to as personalised learning [19].

This article examines the learning process as a subject of research. In this context, the didactic objective is represented as knowledge (Objective — O). The learner's current level of mastery (memorisation, understanding, application, analysis, synthesis, comparative evaluation, proficiency, mastery [5]) is also regarded as knowledge (Comprehension — C).

Knowledge (an internal representation in computer memory) takes the form of a labelled graph, where the vertices correspond to specific facts or generalised concepts, and the edges denote relationships or associations between vertices (ontology, semantic network, frame, etc.) [6]. Hereinafter, when referring to knowledge, unless otherwise specified, we mean a certain graph corresponding to that knowledge.

The quality of learning is represented as a value that semantically denotes proximity, similarity or distance (Distance — D) between the learner's level of mastery C and the learning objective O. For any two given graphs C and O, the distance can be defined [21] on the basis of:

- the maximum distance of common subgraphs;

- structural similarity;
- editing operations;
- kernel functions.

Since we are concerned with knowledge construction, the most suitable method is one based on editing operations, the algorithms for which are described in detail by Aggarwal in Chapter 17 of his textbook [21].

1. Learning Object Repository

A learning object is a representation of a piece of content — an SCO [7] of learning material — together with the metadata required to interact with a learning platform. An SCO may be a basic asset (text, procedure, image, audio, video, etc.) or a combination of basic assets and SCOs. Downs [9] refers to such objects as ‘Learning Objects’.

A learning object repository (R) is a repository containing a collection of learning objects, organised according to metadata. Each learning object in the repository is described by a triplet of the form $\langle \text{Pre}, \text{Exp}, \text{Res} \rangle$, where Pre is the prior knowledge required to understand the explanation, Exp is the explanation itself (learning content), and Res is the knowledge acquired by the learner after studying the object. The repository serves as the basis for the automatic design of didactics: the algorithm for finding a suitable learning object relies on matching the required prior knowledge and the object’s outcome with a given didactic objective.

2. Didactics

We model the concepts of didactics for teaching a particular body of knowledge.

Definition (Didactics): We shall call a didactic approach to learning (hereinafter simply ‘didactics’) the triplet $D = \langle \text{Pre}, \text{Res}, G \rangle$, where Pre is some prior knowledge necessary for learning the given knowledge; Res is the didactic objective of the learning — the knowledge acquired by the learner as a result of learning based on this didactics; $G = \langle N, E \rangle$ — a didactic graph — a directed graph of didactically adapted content. N is the set of vertices of the graph; E is the set of edges of the graph.

The vertices of the didactic graph G correspond to events in the learning process; that is, triples of the form $\langle \text{Pre}, \text{Exp}, \text{Res} \rangle$ are assigned to the vertices, where:

Exp (explanation) — the object of learning;

Pre (Precondition) — a precondition — some prior knowledge required to understand Exp, that is, the explanation is based on this knowledge;

Res (Result) — the postcondition, the result — certain knowledge which the learner will (must) acquire as a result of the explanation of Exp.

The semantics of a vertex means that ‘if the learner knows Pre, then after the explanation of Exp, he or she will know Res’.

The edges of the didactic graph G correspond to ‘follows’ relations amongst the vertices of the graph. If there is an edge from vertex n_i to vertex n_j ,

this means that the explanation $n_j.Exp$ follows from (is based on) the explanation $n_i.Exp$.

For vertex n , $n.Pre \subseteq n.Res$ always holds, and $\Delta n = n.Res - n.Pre$ represents the knowledge acquired by the learner following the explanation $n.Exp$; this is referred to as the didactic effect of vertex n .

We assume that for **every** vertex n in the didactic graph, the following conditions hold:

- 4) $\forall j \in predecessor(n), \neg(n.Pre \subseteq (U_{i \in predecessor(n)} i.Res - \Delta j))$ — the necessity condition for any predecessor;
- 5) $n.Pre \subseteq U_{i \in predecessor(n)} i.Res$ — the sufficiency condition for predecessors;
- 6) $U_{i \in predecessor(n)} i.Res = \min\{\exists x \in \mathfrak{R}(N) : n.Pre \subseteq U_{i \in x} i.Res\}$ — the minimality condition for predecessors.

Here: $predecessor(n)$ — the set of predecessors of vertex n ; $\mathfrak{R}(N)$ — the set of all subsets of the set N .

Condition 1 states that vertex j is necessary for understanding (mastering) $n.Exp$; that is, without mastering the explanation contained in predecessor j , it is not possible to master successor n . Condition 2 states that, in order to master a successor, it is sufficient to master all its predecessors; that is, the predecessors of n are sufficient for mastering the explanation of n . Condition 3 states that the set of a node's predecessors constitutes the minimum amount of knowledge required to master the explanation of its successor; that is, any other set of nodes, the mastery of which is sufficient for mastering n , contains more knowledge than that contained in all of n 's predecessors.

From the perspective of constructing a didactic system, a vertex is attached as a successor to the minimum number of vertices on which it logically depends; at the same time, its predecessors together constitute sufficient knowledge to understand the explanation of vertex n .

The **initial** vertex of a didactic structure is the one that has no predecessors. It may be the case that there are several initial vertices in the didactics — $n_1, n_2, \dots, n_k$. In that case, they can be reduced to a single initial vertex — n_s — by joining $n_1, n_2, \dots, n_k$ as successors of n_s , and setting $n_s.Res = U_{i \in \{n_1, \dots, n_k\}} j.Pre$ with an empty didactic effect ($\Delta n_s = \emptyset$), that is, $n_s.Pre = n_s.Res$. Furthermore, without loss of generality, we assume that the didactic graph G has a unique initial vertex n_s (denoted as $G.n_s$) with the didactic precondition $G.Pre$. Let us call this the didactic precondition $D.Pre$.

A **terminal** vertex in a didactic graph is one that has no successors. Similarly, there may be several terminal vertices $n_1, \dots, n_k$ in a didactic graph. These can also be reduced to a single one — n_f — by connecting $n_1, \dots, n_k$ as predecessors of n_f , and setting $n_f.Pre = U_{i \in \{n_1, \dots, n_k\}} j.Res$ with an empty didactic effect ($\Delta n_f = \emptyset$), that is, $n_f.Res = n_f.Pre$. Furthermore, without loss of generality, we assume that the didactics has a unique finite vertex n_f (denoted as $G.n_f$) with didactic result Res . We denote the didactic result as $D.Res$.

Figure 1 shows a didactic graph in which the four initial vertices (a, b, c, d) are reduced to a single vertex (n_s), and the two terminal vertices (j, k) are reduced to a single vertex (n_f).

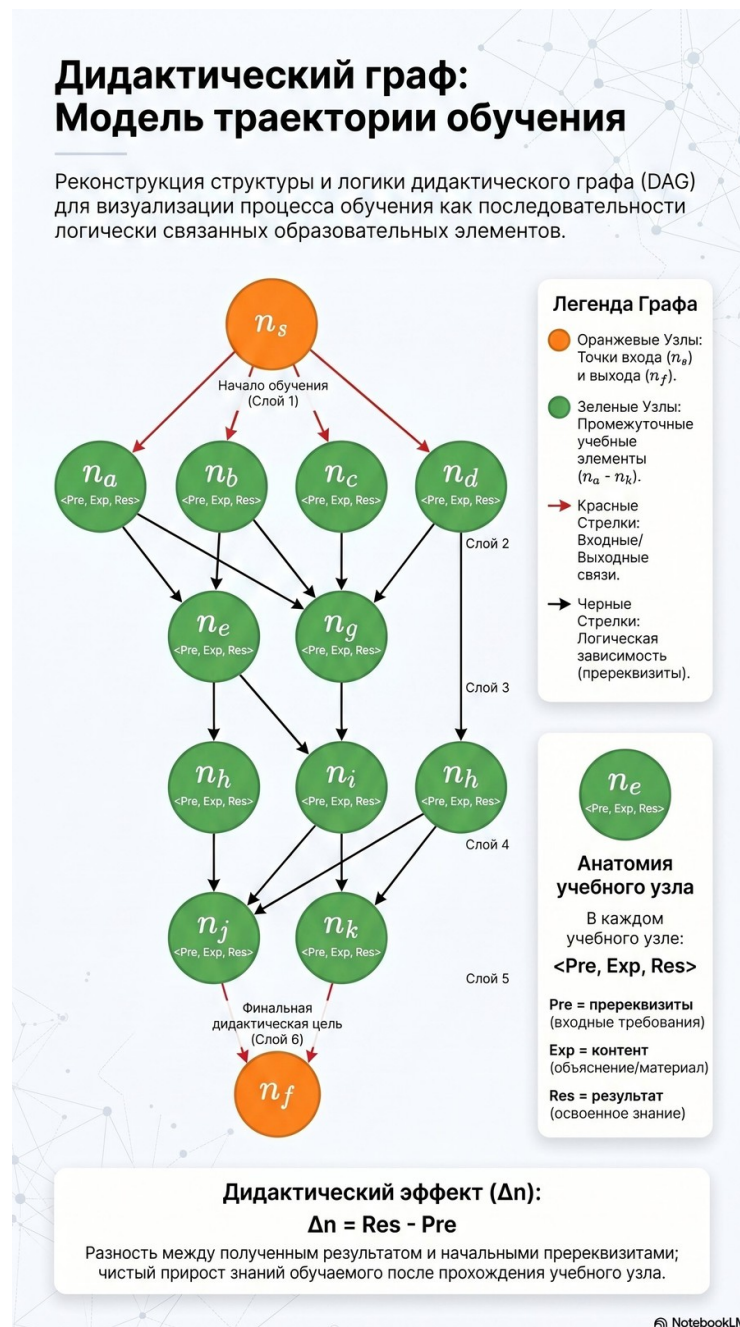


Fig. 1. Didactic graph.

3. Designing didactics

Let us assume that the repository R contains a sufficient number of learning objects for a given learning objective. Let us consider the task of adding a new vertex.

3.1. Determining the predecessors of a vertex

Determining the predecessors of a vertex consists of two algorithms: FindPredecessors and CreatePredecessors. The CreatePredecessors algorithm generates a list of all potential predecessors, PotentialList, and then calls

FindPredecessors to determine the optimal list of predecessors, OptimalList.

FindPredecessors determines a list of predecessors in which the didactic effects do not overlap; that is, there will be no two vertices that repeat a particular piece of knowledge. To achieve this, the variable Δ_{hor} is used, which accumulates the didactic effects of the discovered predecessors. Furthermore, the total knowledge in OptimalList will be minimally covering, i.e. without any superfluous knowledge required to understand the explanation of vertex n . After this, CreatePredecessors creates predecessor vertices for each element in the OptimalList and (if present) an additional vertex for the remaining missing knowledge $Pre - MinRes$, required to understand the explanation at n .

Global variables used in the algorithms:

- A list of all potential predecessors — PotentialList;
- The total knowledge required for the predecessors — Pre;
- Minimum union of predecessor results — MinRes;
- Optimal list of predecessors — OptimalList.

Algorithm FindPredecessors (n , Res, Δ_{hor} , MinRes, CurPred)

Input: The current vertex in the PotentialList — n , the current union of predecessors' results — Res, the current union of predecessors' effects — Δ_{hor} , the current list of predecessors — CurPred.

Output: The optimal list of predecessors — OptimalList.

Method:

$m = \text{Next}(\text{PotentialList}, n);$

If $\Delta_{hor} \cap \Delta_n = \emptyset$ then // Horizontal non-intersection condition: $y =$

$\text{Res} \cup n.\text{Res}; \text{Incl}(\text{CurPred}, n);$

If $y \supseteq \text{Pre}$, then // The number of predecessors is sufficient If y

$\subset \text{MinRes}$, then // Adjust the optimal list

$\text{MinRes} = y; \text{OptimalList} = \text{CurPred};$

End

Else

If $m \neq \text{null}$, then

$\text{FindPredecessors}(m, y, \Delta_{hor} \cup \Delta_n, \text{CurPred});$ // Taking into account the value of n Otherwise

If $S = \emptyset$ then $\text{MinRes} = y; \text{OptimalList} = \text{CurPred};$ End

End

End

$\text{Excl}(\text{CurPred}, n);$

End

If $m \neq \text{null}$, then

FindPredecessors(m , Res , Δ_{hor} , CurPred); // Ignoring version

n All

It is important that the precondition ($n.\text{Pre}$) does not overlap with the explanation; in other words, the explanation must not be based, either directly or indirectly, on itself. To achieve this, the function CreatePredecessors uses the variable Δ_{ver} , which accumulates didactic effects along the path of precedence.

Algorithm CreatePredecessors (n , D , R , Δ_{ver})

Input: The vertex for which predecessors are to be determined — n ; the current (constructed so far) didactic graph — D ; the repository of elementary learning content — R ; the union of didactic effects along the predecessor path — Δ_{ver} .

Output: The set S of all predecessors of vertex n and the missing knowledge m .

Method:

$\text{Pre} = n.\text{Pre}$; $\text{OptimalList} = \emptyset$; $\text{PotentialList} = \emptyset$;

For $\forall d \in B$ (first over D , then over $R-D$), perform: If (Δ_d

$\cap \Delta_{\text{ver}} = \emptyset$) & ($\Delta_d \cap \text{Pre} \neq \emptyset$), then $d \rightarrow \text{PotentialList}$

End

FindPredecessors($\text{PotentialList.first}$, $\emptyset$, $\emptyset$, PotentialList , $\emptyset$);

For all $k \in \text{OptimalList}$, execute

GenPred(n , k);

CreatePredecessors(k , D , R , $\Delta_{\text{ver}} \cup \Delta_k$)

End

If $\text{MinRes} \subset \text{Pre}$, then

$n_s.\text{Res} = n_s.\text{Res} \cup \text{Pre} - \text{MinRes}$;

GenPred(n , n_s);

That's all

Additional functions used:

- Incl(List, n) — adds element n to the list List;
- Excl(List, n) — removes element n from the list List;
- Next(List, n) — returns the element following n in the list List;
- GenPred(n , k) — creates an arc (k , n);
- CreateNode(Pre , Exp , Res) — creates a node with the triplet $\langle \text{Pre}, \text{Exp}, \text{Res} \rangle$;

- $\text{Predecessors}(n)$ — a list of all predecessors of vertex n .

3.2. Didactic Generation

Didactic generation is based on the learning object. The DidacticGen algorithm creates the start and end vertices with an empty didactic effect and generates all predecessors of the end vertex by calling CreatePredecessors.

Alg. DidacticGen (O, R)

Input: A repository of elementary learning content — triples $\langle \text{Pre}, \text{Exp}, \text{Res} \rangle$ — R and a didactic objective (Objective) — O.

Output: Didactic D, where $D.\text{Res} \supseteq O$.

Method:

$n_f = \text{CreateNode}(O, \emptyset, O); D = n_f$

; $n_s = \text{CreateNode}(\emptyset, \emptyset, \emptyset);$

$\text{CreatePredecessors}(n_f, D, R, \emptyset);$

Proposition 1. The didactics constructed by the above algorithm is acyclic.

Proof. Suppose there exists a cycle. Then we can consider a sequence of vertices $e_{i1}, e_{i2}, \dots, e_{ik}$ forming a path such that $e_{i1} = e_{ik}$. Then, since in the CreatePredecessors algorithm the variable Δ_{ver} accumulates didactic effects along the path and does not allow the didactic effect of the next vertex on the path to overlap with Δ_{ver} , it follows that the didactic effect of the vertex is added twice. This contradiction proves the present statement. ■

Subdidactics: A didactics D_1 is a subdidactics of D_2 if $D_1.O \subseteq D_2.O$ and $D_1 \subseteq D_2$ in the graph sense, that is, $N_{D_1} \subseteq N_{D_2}$ & $E_{D_1} \subseteq E_{D_2}$.

Equivalent didactics: Didactics D_1 and D_2 are equivalent if $O_{D_1} = O_{D_2}$ and $D_1.\text{Pre} = D_2.\text{Pre}$.

Detailed vertex: For a vertex n of didactics D_1 , didactics D_2 is a detailing if n and D_2 are equivalent.

Detailed didactics: For given didactics D_1 and D_2 , we say that D_2 details D_1 if there exists at least one vertex in D_1 that D_2 details, whilst the rest of the didactics coincide.

4. Training

At present, generating a didactic is a demanding task (in terms of both time and data volume) for standard servers. Therefore, once a didactic has been generated, it is not usually deleted after use, but is stored in a repository for future reuse. Consequently, when searching for a learning element in the repository, there is a certain probability of finding a ready-made didactic resource.

From time to time, it is necessary to update the stored teaching materials to incorporate newly added learning elements and teaching materials. However, this issue falls outside the scope of this article.

When a didactic is found in the repository whose objective covers the specified one, it is not necessary to construct a didactic from scratch — the one found can be used. We then model the learner's learning process based on the pre-constructed didactic.

It is not necessary for the learner to study all the learning materials assigned to the nodes of the didactic framework if the learning objective is a subset of the objective of the constructed didactic framework. They may not start from the beginning, as they are already familiar with some of the material, or they may not study it all the way through; in other words, their learning objective is a sub-objective of the objective of the given didactic framework. Thus, it will not be necessary to 'work through' the entire didactic framework, but rather a specific sub-didactic framework. The learner studies the explanations of all the vertices of the sub-didactics sequentially in a certain order. Based on condition 2 of the didactics, the learner will master the learning material by first studying all the predecessors of a vertex and then the successor vertex itself. The order in which the predecessors themselves are studied is not important.

Definition (Learning): We call 'learning' such a linearisation (numbering) of the vertices of sub-didactics whereby the above condition is not violated.

Suppose a learner (L) has the goal of acquiring a certain piece of knowledge (L.O) with prior preparation (knowledge) L.C. We assume that $L.C \subset L.O$, otherwise there is no point in the learning process. In order to make use of a

certain constructed didactics D, it is necessary that $(D.Pre \subseteq L.C) \ \& \ (D.O \supseteq L.O)$, that is, the learner must be able to master the didactics' teaching material and, following achieving this, reach their goal. We shall call such a didactic approach suitable for the learner L.

4.1. Definition of the minimum goal satisfying the learner

The algorithm for determining the minimum goal that satisfies the learner, based on the parameters O — the object of learning — and D — the didactics — identifies a vertex n of the didactics D such that $n = \text{Min}(k.Res : \forall k \in D.N) \ \& \ O \subset n.Pre$.

The algorithm

traverses all vertices of the graph D, starting with $D.n_f$, from the bottom upwards through all predecessors, the result of which covers O.

Alg. MinObjective (O, D, m)

Input: An objective (L.O) and a didactics D such that $L.O \subset D.Res$.

Output: A vertex minObjective $\in D.N$ such that $L.O \subseteq \text{minObjective}.Res$, and for $\forall n \in D.N$, where $\text{minObjective} \neq n$: $\text{minObjective}.Res \subseteq n.Res$ or $n.Res \subset L.O$. That is, minObjective is the vertex that minimally covers L.O. The minimal subdidactics $D_{\min}$ that satisfies L.O.

Method: Let Q be some queue. $m = D.n_f$;

Incl(Q, D.n_f);

While $Q \neq \emptyset$, perform

```

n = Excl(Q, n);
If n.Res  $\subset$  minObjective.Res, then minObjective = n; End For
 $\forall k \in$  Predecessors(n), execute
    If k  $\notin$  Q, then Incl(Q, k); End
loop End loop
End

```

Note that the search through a vertex's predecessors should not be terminated here if the vertex's Res does not cover L.O, because Res includes the knowledge $\text{Pre} + \Delta$, and Pre is a subset of Res of its predecessors; thus, there may be a predecessor such that its Res covers L.O.

If, for given L and D, $\text{MinObj}(\text{L.O}, D) = D.n_s$, this means that the given didactics is too complex for the learner in question and requires prior preparation to be learnt.

5. Learning tree

The learner's training in didactics reaches a vertex with a minimum objective that satisfies the learner (covering the learner's objective, $\text{Res} \subseteq \text{L.O}$), and begins with vertices that lead to the minimum objective and are sufficiently simple for the learner to master (covered by the learner's prior knowledge, $\text{Pre} \subseteq \text{L.C}$). Once the minimum learning objective has been defined, the minimum sub-didactics required for learning is determined. It is precisely this minimal sub-didactics that the learner must 'go through' to achieve the objective L.O.

The algorithm for defining the learning tree, LearnTree, defines a certain tree which serves to enable the learner to achieve the learning objective.

Alg. LearnTree (L.C, D, minObjective)

Input: Preliminary preparation L.C, didactics D with the root node minObjective.

Output: A subgraph formed on the basis of the marked vertices of the didactics.

Method: Let Q be a queue. Incl(Q, minObjective);

While $Q \neq \emptyset$, perform

n = Excl(Q, n); Mark n;

For $\forall k \in$ Predecessors(n), do the

following: If ($\text{L.C} \subset k.\text{Pre}$) then

If k $\notin$ Q, then Incl(Q, k);

Otherwise, mark n;

End

All

PostOrder(minObjective);

Here, PostOrder(*n*) is the ascending (left-to-right or right-to-left) numbering [24] of an acyclic graph, starting from vertex *n*. PostOrder numbers the vertices in a manner similar to a tree, with one difference: when it encounters a vertex that has already been numbered, it backtracks. If we remove the edges connecting such vertices, we obtain a tree. Let us call such a tree LearnTree(*L.C*, *D*, minObjective) — a learning tree.

For example, if we number the following acyclic graph using PostOrder, we obtain a tree.

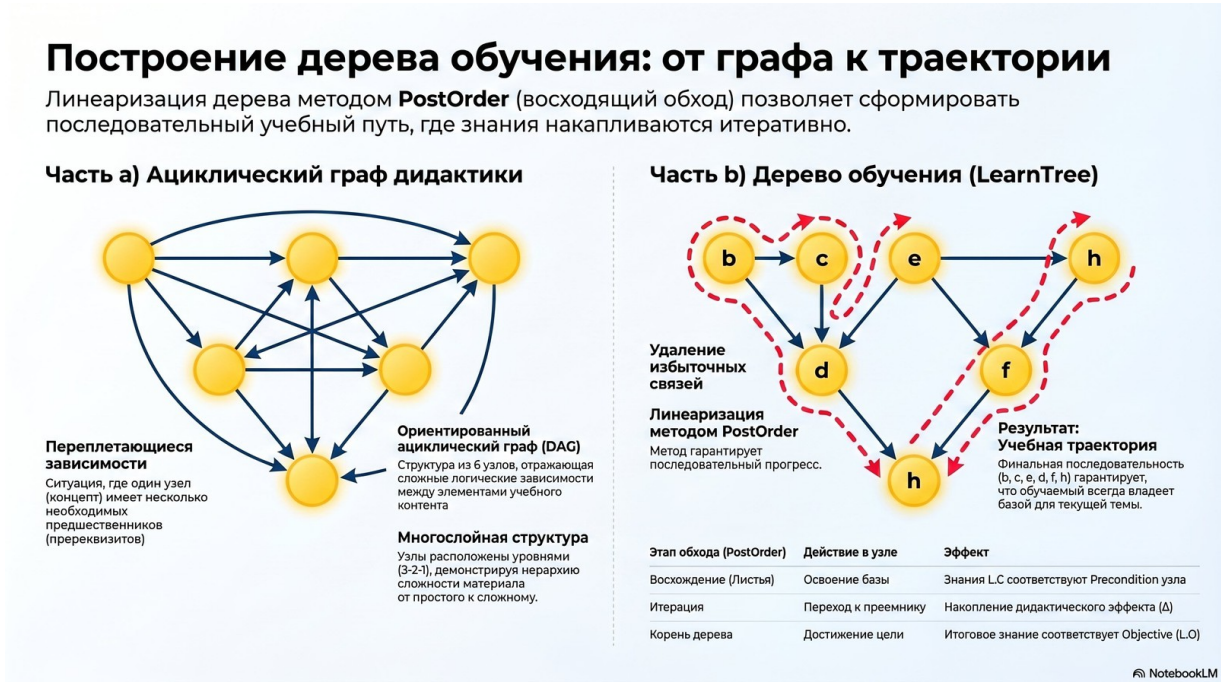


Fig. 2. a) acyclic graph, b) tree obtained by PostOrder numbering.

Statement 2. Linearising a learning tree using PostOrder constitutes a learning process.

Proof. Since PostOrder is an upward-traversal order, the number of a successor will always be greater than that of any predecessor. When a vertex of an acyclic graph is traversed, its predecessors are numbered first, followed by the vertex itself — upon the first visit to it from one of its successors. On subsequent visits via other successors, it is ignored. Thus, every successor has a higher number, which means that PostOrder is a linearisation under which

the learning condition is satisfied. This proves the statement. ■

If the learner studies the material assigned to the vertices of the learning tree by traversing all the vertices of the tree, having first numbered them using PostOrder, then for any vertex *n* of the tree that the learner begins to study, then $L.C \supseteq n.Pre$ holds. This means that the learner does not find themselves in a situation where the current explanation is based on material they have not yet studied.

If we consider the learning tree for a given didactic system — LearnTree(*D.Pre*, *D*, MinObj(*D.O*, *D*)) — then, by linearising (sequentially concatenating) the learning materials

assigned to the vertices of the learning tree in PostOrder, we obtain a didactically revised **e-textbook**.

In this way, the learner achieves their goal by studying the materials assigned to the nodes of the learning tree, traversing all the nodes of the tree in PostOrder, starting from its leaves. In the process, the learner's knowledge accumulates iteratively, node by node: each time they pass through node n , the learner gains an increment Δn . It can be said that this model corresponds to iterative (flexible) learning [26].

References

1. Clark R.C., Mayer R.E. e-Learning and the Science of Instruction. Proven Guidelines for Consumers and Designers of Multimedia Learning. 3rd ed. Pfeiffer, 2011. 527 p.
2. Tchoshanov M. Engineering of Learning: Conceptualising e-Didactics. UNESCO Institute for Information Technologies in Education, Moscow, 2013. 194 pp.
3. Doneva R., Denev D., Totkov G. On the Didactic Principles, Models and E-Learning. International Journal 'Information Theories and Applications'.
4. Brachman R.J., Levesque H.J. *Knowledge Representation and Reasoning*. Morgan Kaufmann, 2004. 381 pp.
5. Bloom B.S., Engelhart M.D., Furst E.J., Hill W.H., Krathwohl D.R. Taxonomy of Educational Objectives: The Classification of Educational Goals. Vol. Handbook I: Cognitive Domain. New York: David McKay Company, 1956.
6. Sowa, J.F. (ed.) Principles of Semantic Networks: Explorations in the Representation of Knowledge. Morgan Kaufmann Publishers, 1991. 576 pp.
7. Lehmann, F. 'Semantic Networks'. Computers in Mathematics and Applications, Vol. 23, Nos. 2–5, pp. 1–50, 1992.
8. Christianson N.H., Blevins A.S., Bassett D.S. Architecture and evolution of semantic networks in mathematics texts. Proc. R. Soc. A 476: 20190741. <http://dx.doi.org/10.1098/rspa.2019.074>
9. Downes S. Learning Objects: Resources For Distance Education Worldwide. International Review of Research in Open and Distance Learning. Vol. 2, No. 1, 2001.
10. Stutt A., Motta E. Semantic Learning Webs. Journal of Interactive Media in Education, Special Issue on the Educational Semantic Web, No. 10 (2004).
11. Devedzic V. Education and the Semantic Web. International Journal of Artificial Intelligence in Education, 14, 39–66 (2004).
12. IEEE LTSC: IEEE Learning Technology Standards Committee Glossary. IEEE P1484.3 Glossary Working Group, draft standard (2001).
13. ADL Initiative. Sharable Content Object Reference Model (SCORM). Advanced Distributed Learning Initiative, 2004. Available online at <http://www.adlnet.org/>
14. Kiv A., Soloviev V., Tarasova E., Koycheva T., Kolesnykova K. Semantic knowledge networks in education. E3S Web of Conferences 166, 10022 (2020) ICSF 2020. <https://doi.org/10.1051/e3sconf/202016610022>

15. Berners-Lee T., Hendler J., Lassila O. The Semantic Web. Scientific American, May 2001.
16. Hodgins W. Out of the past and into the future: Standards for technology-enhanced learning. In: Ehlers U., Pawlowski J. (eds.). Handbook on Quality and Standardisation in E-Learning, pp. 309–327. Springer Berlin, 2006.
17. Tane J., Schmitz C., Stumme G., Staab S., Studer R. The Courseware Watchdog: an ontology-based tool for finding and organising learning material. In: Symposium on Mobile Learning and Research, Kassel, 2003.
18. Advanced Distributed Learning. SCORM 2004 4th Edition Overview. ADL Initiative, 2009.
19. Weimer, M. *Learner-Centred Teaching: Five Key Changes to Practice*. 2nd ed. San Francisco: Jossey-Bass, 2013. 288 pp.
20. Papert, S. and Harel, I. Constructionism. Ablex Publishing Corporation, 1991.
21. Aggarwal C.C. Data Mining: The Textbook. Springer, 2015. 746 pp.
22. Tan P., Steinbach M., Karpatne A., Kumar V. Introduction to Data Mining. 2nd ed. New York: Pearson Education, 2019. 1459 pp.
23. Bloom B.S. et al. — see [5].
24. Aho A.V., Hopcroft J.E., Ullman J.D. Data Structures and Algorithms. Addison-Wesley, 1983.
25. Piaget J. The Equilibration of Cognitive Structures: The Central Problem of Intellectual Development. University of Chicago Press, 1985.
26. Sharples M. et al. Agile Methods for Technology-Enhanced Learning. In: Proceedings of the European Conference on Technology-Enhanced Learning, 2014.